

CampusEats

CS543 – Assignment 2

Service Design, Contracts & Validation

Mohammed Sahil Siddiqui – 20252651055

Ritik Kumar Sinha – 20252651043

Jeeshan Adil – 20252651026

Kruti Kansagara – 20252651027

August 16, 2026

Contents

1 Team Members and Service Responsibility 2

2 Task 1 – Capabilities 2

3 Task 2 – Service Design 2

4 Task 3 – Service Contracts 3

4.1	Contract: Orders (main service)	3
4.2	Contract: Catalogue (main service)	3
4.3	Contract: Accounts	4
4.4	Contract: Payments	4
4.5	Contract: Delivery	4
4.6	Contract: Notifications	4

5 Task 4 – Full Specification: placeOrder 4

6 Task 5 – Database Schema 5

7 Task 6 – Validation 6

8 Conclusion 7

1 Team Members and Service Responsibility

Member	Roll No.	Role	Service Responsibility
Mohammed Sahil Siddiqui	20252651055	Team Leader	Orders, Payments
Ritik Kumar Sinha	20252651043	Team Member	Catalogue
Jeeshan Adil	20252651043	Team Member	Accounts, Delivery
Kruti Kansagara	20252651027	Team Member	Notifications

2 Task 1 – Capabilities

CampusEats has six main jobs, called capabilities. Each capability represents one clear type of work in the system.

#	Capability	What it does	Service
1	Manage profile and delivery addresses	Students can create and manage their profile and the addresses where food should be delivered.	Accounts
2	Browse restaurants and menus	Students can view available restaurants, menus, food items and prices.	Catalogue
3	Add items to cart and place orders	Students can add food to a cart and place an order.	Orders
4	Process payments and refunds	Students can pay for an order and receive a refund when required.	Payments
5	Assign a rider and track delivery	A rider is assigned to an order and the student can follow delivery progress.	Delivery
6	Send order notifications	Students receive updates when an order is placed, paid, on the way, or delivered.	Notifications

3 Task 2 – Service Design

The service design contains six services. Each service owns its own data and other services use its contract instead of directly accessing its internal data.

- **Accounts** – owns users and addresses – Jeeshan Adil.
- **Catalogue** – owns restaurants, menus and prices – Ritik Kumar Sinha.
- **Orders** – owns carts, orders and order status – Mohammed Sahil Siddiqui.
- **Payments** – owns transactions and refunds – Mohammed Sahil Siddiqui.
- **Delivery** – owns riders and delivery assignments – Jeeshan Adil.
- **Notifications** – owns the notification/message log – Kruti Kansagara.

CampusEats — Service Design

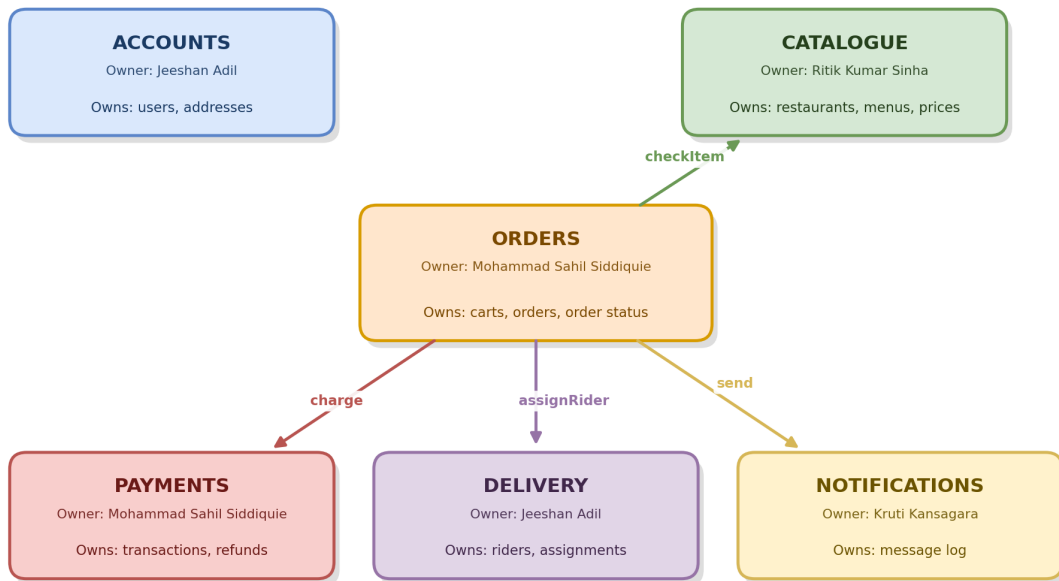


Figure 1: CampusEats service design

4 Task 3 – Service Contracts

A service contract describes how another service can use a service. It shows the operation, input, output and possible errors. Internal tables, ID formats and SQL are not exposed through the contract.

4.1 Contract: Orders (main service)

Operation	Input	Output	Errors
addToCart	studentId, itemId, qty	cart	item unavailable
placeOrder	studentId, items, deliveryAddressId, paymentMethodId	orderId, status, total, estimatedMinutes	empty cart, bad address, item unavailable, payment declined
getOrder	orderId	order + status	not found
cancelOrder	orderId	status: cancelled	too late to cancel

4.2 Contract: Catalogue (main service)

Operation	Input	Output	Errors
listRestaurants	area / filter	restaurants[]	service unavailable
getMenu	restaurantId	menu items + prices	restaurant not found
checkItem	itemId	available?, price	item not found

4.3 Contract: Accounts

Operation	Input	Output	Errors
login	email, password	sessionToken, userId	invalid credentials
manageProfile	userId, updates	updated profile	invalid field
getAddress	addressId	address	not found

4.4 Contract: Payments

Operation	Input	Output	Errors
charge	orderId, amount, paymentMethodId	receipt, transactionId	payment declined
refund	transactionId, amount, reason	refundId, status	transaction not found, already refunded

4.5 Contract: Delivery

Operation	Input	Output	Errors
assignRider	orderId, addressId	assignmentId, rider-Name, eta	no rider available
trackDelivery	orderId	riderLocation, status	not found

4.6 Contract: Notifications

Operation	Input	Output	Errors
send	userId, type, message	messageId, status: sent	invalid user

5 Task 4 – Full Specification: placeOrder

`placeOrder` is the central operation of CampusEats. It takes a student's selected food items and turns them into an order. The Orders service controls the process and uses the contracts of other services when it needs information or actions owned by them.

Input

- `studentId`
- `items: [itemId, qty]`
- `deliveryAddressId`
- `paymentMethodId`

Output

- `orderId`
- `status: placed`
- `total`
- `estimatedMinutes`

Error cases

- item unavailable
- payment declined
- invalid address
- empty cart

Details hidden from callers

The caller does not need to know:

- how orders and order items are stored internally;
- how the order ID is created;
- which rider is selected or how the rider is selected;
- which payment gateway is used inside `charge()`;
- how the total price is calculated internally.

Flow

1. The student sends a `placeOrder` request to Orders.
2. Orders asks Catalogue through `checkItem` to check availability and obtain the price.
3. Orders asks Payments through `charge` to process the payment.
4. Orders asks Delivery through `assignRider` to arrange delivery.
5. Orders asks Notifications through `send` to notify the student.
6. Orders returns the order confirmation to the student.

Orders controls the overall process and owns its own carts, orders and order status. It obtains information belonging to other services through their contracts rather than directly accessing their internal data.

6 Task 5 – Database Schema

The database design follows the boundary rule that each table belongs to one service only. The ER diagram and SQL sketch are submitted separately as required.

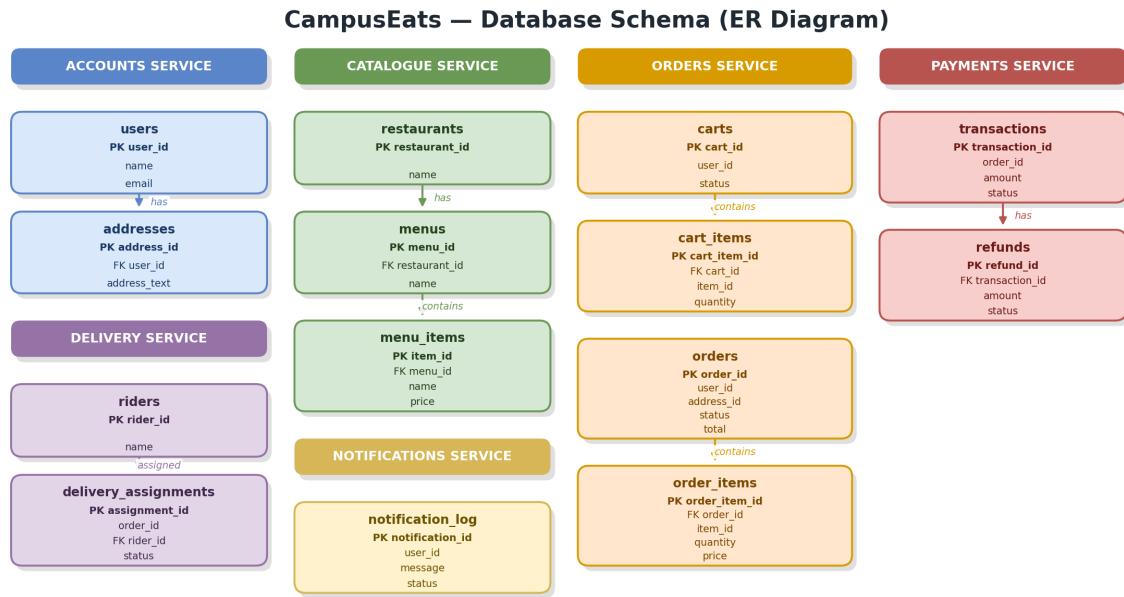


Figure 2: CampusEats database ER diagram

7 Task 6 – Validation

Each service is checked against five properties: reachable over a network, self-contained, has a clear contract, independent, and loosely coupled.

Service	Reachable	Self-contained	Contract	Independent	Loosely coupled
Accounts	✓	✓	✓	✓	✓
Catalogue	✓	✓	✓	✓	✓
Orders	✓	✓	✓	✓	✓
Payments	✓	✓	✓	✓	✓
Delivery	✓	✓	✓	✓	✓
Notifications	✓	✓	✓	✓	✓

Notes

Delivery – Independent: Delivery may depend on an outside maps or routing service when selecting a rider. If that outside service becomes unavailable, rider assignment could be affected. A suitable fix is to keep the outside dependency behind Delivery's own contract, use a timeout, and provide a backup option such as selecting an available rider manually.

Notifications – Contract: Notifications currently exposes the `send` operation, which is enough for the current design. If the system later needs to check whether a notification was delivered, an operation such as `getStatus(messageId)` can be added.

Overall, the six services can be reached over a network, keep their own data, expose contracts, and communicate through defined operations rather than sharing databases.

8 Conclusion

The CampusEats design starts from the system capabilities and groups related work into six services. Each service owns its data and exposes a small contract to the other services. The central `placeOrder` operation coordinates Catalogue, Payments, Delivery and Notifications without exposing their internal implementation details. This keeps the service boundaries clear and reduces direct coupling between services.